1. What is a Web Application?

A web application is an application that users access through a web browser. When a user performs an action, the browser sends a request, the application processes it, and the required result is returned to the user. A web application is made up of multiple components that work together to process requests and generate responses. Understanding how a web application works is important before learning how to build one using backend technologies such as Java.

2. What are the three tiers of a Web Application?

Every web application consists of three tiers: **Frontend**, **Backend**, and **Database**.

The **Frontend** is the part of the application that users can see and interact with. The **Backend** performs all the processing and executes the application's logic before sending the result back to the user. The **Database** stores the application's data permanently. These three tiers work together to form a complete web application. Enterprise applications also follow the same three-tier architecture.

3. Which technologies are used in each tier of a Web Application?

Different technologies are used to build different tiers of a web application.

The **Frontend** can be developed using **HTML, CSS, JavaScript, and React**. The **Backend** can be developed using programming languages such as **Java, Python, Go, or C#**. The **Database** layer can use relational databases like **MySQL, Oracle, and PostgreSQL**, or non-relational databases like **MongoDB**. Depending on the application's requirements, other databases such as vector databases can also be used.

4. Why do we write programs?

Programs are written to automate tasks. Automation means that instead of a human performing a task manually, the computer performs the task automatically. To make the computer perform any task, instructions must be given to it. Writing these instructions in a programming language is called programming. Every program is created to automate one or more tasks that the computer can execute.

5. How can a computer be instructed to print output in Java?

To print something on the monitor in Java, the statement **System.out.println()** is used. The content that needs to be displayed is written inside double quotation marks. Every Java statement ends with a semicolon. Once this statement is written correctly, the computer knows what needs to be printed. However, this

BTM Layout 2nd Stage,
Bengaluru, Karnataka 560076 8050749191

statement alone is not sufficient for execution because it must be placed within the proper structure of a Java program.

6. Why is the main() method important in a Java program?

Every Java program starts executing from the **main()** method. It acts as the predefined starting point of program execution. Whenever the computer runs a Java program, it first looks for the main() method and begins reading the instructions from there. Therefore, every instruction that needs to be executed must be placed inside the main() method. Without it, the program cannot begin execution.

7. Why should every method be placed inside a class in Java?

Java is an Object-Oriented Programming Language. Because of this, every method must be written inside a class. After creating the main() method, it should be enclosed within a class boundary. A class can be given any valid name. Once the class is created, all the program logic and methods are written inside it, forming the basic structure of a Java program.

8. What is the basic structure of a Java program?

The basic structure of a Java program consists of three main parts. First, a **class** is created. Inside the class, the **main()** method is written because it is the starting point of execution. Finally, the program logic or instructions, such as System.out.println(), are written inside the main() method. Once these three parts are present, the basic structure of the Java program is complete and ready for the next stages of execution.

9. What is the difference between RAM and Hard Disk?

A computer contains two important types of memory: **RAM** and **Hard Disk**.

RAM is the primary memory of the computer and is volatile in nature. It stores data only while power is available. If power is removed, all the data stored in RAM is lost. The Hard Disk is secondary memory and stores data permanently. Files remain available in the Hard Disk even after the computer is switched off. During program execution, the processor works with data available in RAM, while the Hard Disk is used for permanent storage.

10. What is Saving?

Saving is the process of transferring data from **RAM** to the **Hard Disk** for permanent storage. When a Java program is typed, it initially exists only in RAM. Since RAM is temporary memory, the program will be lost if power is removed. To permanently store the program, it must be saved using **Ctrl + S** (or **Command + S**

BTM Layout 2nd Stage,
Bengaluru, Karnataka 560076 8050749191

on macOS). During this process, a file name is assigned, and the program becomes a file stored in the Hard Disk.

11. Why can't a Java program be executed immediately after it is written?

After writing a Java program, it exists only in RAM and is written in a high-level programming language. The processor cannot execute it in this form because it understands only machine-level language. The program must first be saved to the Hard Disk, compiled into bytecode, and then converted into machine-level language before it can be executed. Therefore, simply writing a Java program is not sufficient for execution.

12. What is a High-Level Language?

A high-level language is a programming language used by developers to write programs in a simple and understandable form. Java programs are written in a high-level language, making them easier to read and develop. However, the processor cannot understand high-level language directly. Therefore, the program must be converted into another form before it can be executed.

13. What is Compilation?

Compilation is the process of converting a Java program written in a high-level language into **Bytecode**. This conversion is performed using the Java Compiler. During compilation, every instruction in the Java source file is converted into bytecode, preparing the program for the next stage of execution.

14. What is the role of the Java Compiler?

The Java Compiler is responsible for converting a Java source program into bytecode. The source file is provided to the compiler, which processes every instruction and generates a new file containing bytecode. This generated file is used in the next stage of program execution and is necessary before the program can be executed.

15. What is Bytecode?

Bytecode is the code generated after compiling a Java source program. It is an intermediate form of code that is produced by the Java Compiler. Bytecode is not machine-level language, so it cannot be executed directly by the processor. It must first be converted into machine-level language before execution can take place.

BTM Layout 2nd Stage,
Bengaluru, Karnataka 560076 8050749191

16. What is a Source File in Java?

A source file is the file that contains the Java program written by the programmer. It is saved with the **.java** extension and contains the high-level language code. This source file is given to the Java Compiler during compilation, which converts it into bytecode for further execution.

17. What is a Class File in Java?

A class file is the file generated after compiling a Java source file. It contains the bytecode produced by the Java Compiler. The class file has the same name as the source file but uses the **.class** extension. This file is later processed during program execution to produce the final output.

18. Why does the compiled file use the .class extension?

After compilation, the Java Compiler creates a new file containing bytecode. Since this file is different from the original source file, it is stored with the **.class** extension while retaining the same file name. For example, if the source file is **Demo.java**, the compiled file becomes **Demo.class**. This class file is used in the subsequent stages of Java program execution.

19. Why can't the processor execute Bytecode directly?

Although bytecode is generated after compilation, it still cannot be executed directly by the processor because the processor understands only machine-level language. Before execution, the bytecode must be converted into machine-level language. Only after this conversion can the processor read the instructions and perform the required operations.

20. What is the role of the Java Virtual Machine (JVM)?

The Java Virtual Machine (JVM) is responsible for processing the bytecode generated by the Java Compiler. It converts the bytecode into machine-level language so that it can be understood by the processor. After the conversion, the machine-level instructions are loaded into RAM, where the processor executes them and produces the required output. The JVM plays an important role in the execution of every Java program.

21. What is the role of the JIT (Just-In-Time) Compiler?

The **JIT (Just-In-Time) Compiler** is present inside the **Java Virtual Machine (JVM)**. Its responsibility is to convert **Bytecode** into **Machine-Level Language**. Since the processor cannot understand bytecode directly, the JIT Compiler performs this conversion before execution. Once the conversion is completed, the machine-level instructions are ready to be executed by the processor.

BTM Layout 2nd Stage,
Bengaluru, Karnataka 560076 8050749191

22. What is Machine-Level Language?

Machine-Level Language is the language that the processor understands and executes directly. Although programmers write programs in Java, the processor cannot execute Java code or bytecode. Before execution, the program must be converted into machine-level language. Only after this conversion can the processor read the instructions and perform the required task.

23. What is Loading?

Loading is the process of transferring the required program from the **Hard Disk** into **RAM** for execution. A program stored in the Hard Disk cannot be executed directly because the processor works only with data available in RAM. After the machine-level language code is generated, it is loaded into RAM, from where the processor executes it.

24. Why are programs loaded into RAM before execution?

Programs are loaded into RAM because the processor executes instructions that are available in RAM. Even though the program is permanently stored in the Hard Disk, it must first be brought into RAM before execution can begin. Once the required instructions are loaded into RAM, the processor reads them and performs the specified operations.

25. What is the role of the processor during Java program execution?

The processor performs the final execution of a Java program. After the Java program has been converted into machine-level language and loaded into RAM, the processor reads each instruction and executes it. Based on the instructions provided, it performs the required task, such as printing output on the monitor. The processor executes only machine-level instructions and does not directly execute Java source code or bytecode.

26. How does a Java program produce the final output?

A Java program produces the final output through a sequence of steps. The program is first written and saved as a .java file. It is then compiled into bytecode by the Java Compiler. The JVM processes the bytecode, and the JIT Compiler converts it into machine-level language. The converted instructions are loaded into RAM, where the processor executes them. After execution, the processor produces the required output, such as displaying text on the monitor.

27. What is the purpose of the JDK?

The **JDK (Java Development Kit)** provides the tools required to develop and compile Java programs. After installing the JDK, the Java Compiler becomes

BTM Layout 2nd Stage,
Bengaluru, Karnataka 560076 8050749191

available on the system. The transcript demonstrates that the compiler can be found inside the JDK installation directory, and it is this compiler that is used to compile Java source files into bytecode.

28. What is javac?

javac is the **Java Compiler** provided as part of the JDK. It is used to compile Java source files. When a .java file is given to javac, it converts the high-level Java program into bytecode and generates a corresponding .class file. This compiled file is then used during program execution.

29. What is the relationship between a .java file and a .class file?

A .java file is the source file written by the programmer and contains the Java program in high-level language. During compilation, this file is converted into a .class file by the Java Compiler. Both files have the same name, but their extensions are different. The .java file contains the source code, while the .class file contains the generated bytecode that is used during execution.

30. Explain the complete execution flow of a Java program.

The execution of a Java program begins by writing the program in a source file with the .java extension. Initially, the program exists in RAM and is then saved permanently to the Hard Disk. During compilation, the Java Compiler converts the source code into bytecode and creates a .class file. The JVM then processes the bytecode, and the JIT Compiler converts it into machine-level language. The generated machine-level instructions are loaded into RAM, where the processor executes them. Finally, the processor performs the requested operation and produces the required output. This sequence represents the complete execution flow of a Java program.

BTM Layout 2nd Stage,
Bengaluru, Karnataka 560076 8050749191